Bailout Protocol: Mandatory Human Accountability in Multi-Agent Systems

Brodi Blanco[1,2]

[1]Bxthre3 Inc [2]San Luis Valley Research, Monte Vista, CO 81144

Abstract

The deployment of autonomous multi-agent AI systems into high-stakes operational environments has outpaced the development of accountability infrastructure capable of governing them. When a network of AI agents coordinates to decompose and execute complex directives — spawning sub-agents, delegating subtasks laterally, and operating asynchronously under partial observability — the chain of causal responsibility between an outcome and the original human principal becomes mediated by multiple machine actors in ways that are opaque, non-linear, and legally ambiguous. This paper presents the Bailout Protocol: a mandatory architectural primitive that forces agentic systems to halt before entering irreversible states and to obtain explicit, attested human authorization before proceeding. The protocol is defined through a formal state machine model, its trigger condition is expressed in terms of observational boundary violation, and its enforcement is guaranteed by the Fact Layer pre-commit hook and the append-only Forensic Ledger. We prove the protocol's safety, liveness, and accountability properties and demonstrate its integration with the BX3 three-layer framework. The Bailout Protocol is the runtime expression of BX3's upstream accountability guarantee: it makes the framework's static accountability commitments structurally operative under actual exception conditions.

Keywords: multi-agent AI, accountability, human-in-the-loop, fail-safe, autonomous systems, BX3 Framework

1 Introduction

The deployment of autonomous multi-agent systems across critical infrastructure — irrigation networks, financial markets, healthcare logistics, and precision agriculture — has exposed a structural gap in the prevailing architecture of accountability. Modern autonomous agents decompose complex goals into recursive sub-tasks, delegate authority across hierarchies of machine actors, and execute without ongoing human oversight. The prevailing architecture provides no mandatory mechanism for routing exceptional conditions to a human decision-maker when all machine resolution paths have failed. The result is a class of failure modes in which autonomous systems proceed to act without accountability — not because they are adversarial, but because the architecture contains no path back to human consciousness when it is genuinely needed. We call this condition the accountability gap, and the Bailout Protocol is our proposed solution.

1.1 Why Autonomous Multi-Agent Systems Require a Mandatory Bail-Out Mechanism

Consider the structure of a deployed autonomous multi-agent system operating under the BX3Framework. A Human Accountability Anchor at the root authorizes a *Purpose Layer*— a bounded objective with explicit constraints. The *Bounds Engine*engine decomposes that *Purpose Layer*into sub-tasks, spawning child nodes recursively. Child nodes further decompose and spawn, creating a spawning hierarchy of machine actors that spans from the root anchor to leaf-level actuators. The delegation chain is, in principle, the accountability chain: a human at the top authorizes objectives, intermediate agents translate those objectives into operational decisions, leaf agents execute.

In practice, the chain terminates at the first unresolvable exception. When a leaf agent encounters a condition it cannot resolve with certainty — an out-of-range sensor reading, a safety-envelope violation that its own sandbox simulation cannot rule out, a decision whose implications exceed its provisioned legal or operational authority — it does not escalate to a human. It either fails silently, applies a local heuristic that may or may not be appropriate, or propagates the error upward in a form no human can act on coherently. The exception is absorbed by the machine hierarchy. The human at the top of the chain is never notified. The *Purpose Layer*layer anchor — the very entity that holds accountability — is structurally bypassed by the autonomous action.

The mandatory bail-out mechanism we propose is not a "big red button." It is an architectural protocol embedded in every node of the spawning hierarchy such that any unresolvable condition at any depth propagates asynchronously to a Human Accountability Anchor, bypassing all intermediate machine actors, with complete diagnostic context from the Forensic Ledger. The protocol is structurally mandatory: it cannot be suppressed by a compromised node, cannot be bypassed by a machine actor in the resolution chain, and cannot be silenced by a network partition without this silence being detected and logged. The structural mandatory property is the essential contribution of this work.

Existing approaches to exception handling in multi-agent systems treat escalation as a design choice to be made at deployment. If the system designer anticipated a condition, they may have included a human review step; if they did not, the condition is handled by the machine hierarchy alone. The Bailout Protocol removes this dependence on anticipation. Any condition that the *Bounds Engine*engine cannot resolve within its current authority — whether the specific failure mode was anticipated or not — triggers the same mandatory escalation to the *Purpose Layer*layer. The three trigger conditions that govern this escalation — Capability Boundary, Safety Envelope Violation (predicted), and Accountability Boundary — are defined precisely in Section **??**, but the critical property is universality: the protocol fires on unresolvable uncertainty, not on a predefined list of error codes.

1.2 What Happens When Systems Do Not Have One

The absence of a mandatory bail-out mechanism does not produce neutral outcomes. It produces a specific failure pattern we term *autonomous drift*: the progressive extension of machine authority beyond its intended bounds, without human review, until a failure forces human intervention under suboptimal conditions.

Autonomous drift is enabled by the confidence-threshold model that dominates existing human-in-the-loop frameworks. In this model, a human reviews an agent's action only when the agent's confidence falls below a configured threshold. The model assumes that an agent's self-reported confidence is a reliable signal of whether the action is correct with respect to the user's actual objectives. This assumption fails systematically for large language model–based agents, whose confidence scores are calibrated against the model's training objective (next-token prediction), not against the operational objective (correct action in context). A model can express high confidence while being wrong about what the user intended — and the confidence-threshold model will permit the action to proceed without human review.

The consequences of autonomous drift are compounding. When an agent acts without human review and the action produces an acceptable immediate outcome, the system reinforces the pattern: this agent handled this condition autonomously, no escalation was necessary, the confidence threshold was appropriate. Over successive cycles, machine authority extends into territory that was never authorized, never reviewed, and never recorded in any audit log. The *Fact Layer*layer — the deterministic enforcement interface at the actuator boundary — executes actions whose upstream authorization has silently expired. The eventual failure is catastrophic in a specific sense: it occurs under conditions the human never reviewed, about whose context they have no diagnostic record, and whose resolution requires intervention in a physical state that has already diverged significantly from the intended state. The human is not a backup safety layer — they are a last-resort responder called in after damage has propagated. This is the inverse of fail-safe design.

The regulatory environment is beginning to respond to this pattern. The EU AI Act and the NIST AI Risk Management Framework both mandate human oversight for high-risk AI systems as a compliance obligation, not a best practice. These mandates presuppose an architecture capable of satisfying them. A system that relies on confidence thresholds and post-hoc audit logs cannot satisfy the mandate because it cannot guarantee that every unresolvable state reached a human before consequential action was taken. The accountability gap is not merely a technical failure — it is a compliance liability.

1.3 The Core Insight: Accountability Must Have an Architectural Fallback

The core insight of this paper is that accountability cannot be a property of a system's governance documents — it must be a property of its architecture. Specifically, human accountability must be implemented as a structurally mandatory fallback, not as a configurable threshold or an organizational policy.

This distinction matters because governance documents and architectural constraints have fundamentally different failure modes. A governance document can be updated, overridden, or ignored by a system operator. An architectural constraint cannot be bypassed without modifying the system's structure, and any such modification is itself logged by the Forensic Ledger. When accountability is implemented as a governance document, it exists at the discretion of the system operator. When accountability is implemented as an architectural fallback — as the BX3Framework implements it through the Bailout Protocol — it exists independent of any individual operator's choices, including the choices of the humans

nominally responsible for the system.

The BX3Framework implements this through a three-layer separation of concerns. The *Purpose Layer*layer owns all Human Accountability Anchors — the named humans with organizational authority who are the terminal recipients of every bailout escalation. The *Bounds Engine*engine proposes actions within the constraints authorized by the *Purpose Layer*layer. The *Fact Layer*layer enforces deterministic physical and operational constraints at the actuator interface. The Bailout Protocol connects these layers: it guarantees that any state the *Bounds Engine*engine cannot resolve within its current authority — any condition that exceeds the bounds of its provisioned decision space — escalates to a *Purpose Layer*layer anchor, bypassing all intermediate machine actors, with complete diagnostic context drawn from the Forensic Ledger.

The architectural fallback property has a precise meaning in the BX3context. A fallback is not a backup plan that operates when the primary plan fails. It is a mandatory termination condition that the system is structurally required to reach before certain classes of action can proceed. In the BX3architecture, the termination condition is a Human Accountability Anchor, and the mandatory property is enforced by two structural constraints: the immutability of parent pointers in the spawning hierarchy, and the bypass mechanism of the Bailout Protocol itself. No agent can modify its own parent pointer. No machine actor in the resolution chain can redirect an escalation away from its designated Human Accountability Anchor without this redirection being detected and logged as an anomaly. The accountability chain does not rely on the good judgment of system operators. It is structurally enforced.

This is the guarantee that no existing multi-agent framework provides. Existing systems implement exception handling as a design choice, confidence thresholds as a configurable parameter, and human oversight as an organizational policy. The BX3Framework implements human oversight as a mathematical consequence of immutable parent pointers, depth-bounded spawning, and the mandatory termination property of the Bailout Protocol.

1.4 Paper Roadmap

The remainder of this paper is organized as follows. Section 2 reviews the foundational threads converging in the Bailout Protocol: the principle of separation of concerns as an accountability architecture, the problem of accountability in multi-agent AI systems, human-in-the-loop design and its structural limitations, and fail-safe design in safety-critical systems. Section **??** presents the formal design of the Bailout Protocol, including its three triggering conditions, escalation state machine, and the bypass mechanism that severs all machine actors upon activation. Section **??** describes the Forensic Ledger as the audit substrate on which the Bailout Protocol operates, including the 9-Plane Data Action Protocol structure and the hash-chained append-only architecture that guarantees log integrity. Section **??** establishes the protocol's correctness properties: bypass soundness, no suppression by compromised nodes, and deterministic resolution at the Human Accountability Anchor. Section **??** situates the Bailout Protocol within the broader BX3Framework, including its relationship to the Training Wheels Protocol and the Human Accountability Anchor specification. Section 6 acknowledges the protocol's limitations — human response latency under high-frequency events, adversarial conditions, deployment depth scalability, and exploitation

attack vectors — and proposes directions for future work. Section 9 concludes with a discussion of the protocol's implications for mandatory human accountability in autonomous systems and its relationship to emerging regulatory frameworks.

2 Background

The problem of accountability in autonomous systems is not new, but its urgency has escalated sharply with the deployment of multi-agent AI architectures in high-stakes environments. This section reviews four foundational threads that converge in the design of the Bailout Protocol: the principle of separation of concerns as a basis for architectural accountability; accountability as a first-class property in multi-agent AI systems; human-in-the-loop design as the dominant paradigm for maintaining human oversight; and fail-safe design as established in safety-critical systems engineering. We then situate the BX3 Framework's three-layer model as an architecture that synthesizes these threads into a unified accountability design.

2.1 Separation of Concerns and Accountability Architecture

The conceptual foundation for treating accountability as an architectural property rather than a governance afterthought traces to Dijkstra's articulation of the *separation of concerns* principle. In **?**, Dijkstra argued that sound system design requires deliberate boundaries between functional domains, such that each concern can be reasoned about and enforced independently. When concerns are entangled, failures in one domain propagate unpredictably into others, making responsibility unassignable and remediation impossible. Dijkstra's insight, originally applied to software module boundaries, extends naturally to the boundary between autonomous machine action and human accountability. If a machine actor can modify the conditions under which it is evaluated, or if the boundary between machine and human authority is fluid, accountability dissolves. The separation of concerns demands that the authority to set objectives, the authority to propose actions, and the authority to execute physical operations occupy distinct, immutable layers of the system architecture. The BX3 Framework takes this principle as its foundational commitment: each layer has a fixed role, and no layer can unilaterally modify the constraints of another.

2.2 Accountability in Multi-Agent AI Systems

Classical multi-agent systems (MAS) research established that when multiple autonomous actors operate concurrently, attributing responsibility for collective outcomes requires explicit architectural support. As surveyed in **?**, the challenge is fundamental: agents are defined by their autonomy — the capacity to act without direct human instruction — which is precisely the property that severs the human–action chain needed for accountability. In traditional MAS, accountability has been addressed through role assignment (agents play defined roles with associated duties), through voting or consensus protocols (collective decisions distribute or obscure individual responsibility), and through organizational norms (informal

or formal conventions about who is responsible for what). Each of these approaches is susceptible to the *accountability gap*: the scenario in which an autonomous action produces a harmful outcome but no human actor can reconstruct, explain, or be held responsible for the decision chain that led there.

Modern AI agent frameworks have intensified this problem. Large language model–based agents operate probabilistically rather than deterministically, making the input–output relationship non-invertible and the decision logic opaque even to their designers. When multiple such agents are composed in orchestration pipelines, the accountability chain fractures across model boundaries, prompt contexts, and tool invocations that no single human reviews before execution. Recent regulatory frameworks, including the EU AI Act and the NIST AI Risk Management Framework, now require that high-risk AI systems maintain "human oversight" and "decision provenance" not as best practices but as mandatory compliance obligations **?**. These mandates presuppose an architecture that can satisfy them — a structure in which every autonomous action either originates from or can be traced to a human accountability anchor. The Bailout Protocol is designed as that architectural substrate.

2.3 Human-in-the-Loop Design

Human-in-the-loop (HITL) is the dominant paradigm for incorporating human oversight into autonomous systems. In HITL design, a human operator sits at one or more points in the control loop, reviewing, approving, or correcting agent actions before or after execution. The paradigm exists on a spectrum: at one end, humans approve every action in a strict workflow gate; at the other, humans are notified post-hoc of actions already taken, retaining only the ability to investigate and remediate rather than prevent.

The fundamental tension in HITL design is between the efficiency of automation and the fidelity of human oversight. Pure automation achieves speed and scale but sacrifices accountability; full human review preserves accountability but negates the autonomy that makes multi-agent systems valuable. Existing HITL frameworks address this tension through confidence thresholds — autonomous action proceeds when the system's confidence exceeds a configurable level, and human review is triggered only below that threshold. However, this approach has a structural flaw in the context of LLM-based agents: the confidence score produced by a model is not a reliable indicator of correctness with respect to the user's actual objectives, because the model's training objective (next-token prediction) is decoupled from the operational objective (correct action in context). A model can express high confidence while being wrong about what the user actually intended.

The BX3 Framework inverts this logic. Rather than triggering human review based on a machine-generated confidence signal, the Bailout Protocol treats human review as the *default state* for any unresolvable condition, and uses machine confidence only as a signal internal to the machine layers. If the Bounds Engine cannot produce a proposal that satisfies all constraints within its authority, it does not guess — it escalates unconditionally. This architectural commitment eliminates the failure mode in which a highly confident but incorrect agent proceeds to harmful action because no threshold was crossed.

HITL design also confronts the *abstraction leakage* problem **?**: when a human operator is inserted into a running system to investigate or override an agent's action, the operator often loses the contextual information needed to make an informed decision. The hierarchical

context of the agent's reasoning, the state of the broader multi-agent system, and the chain of prior decisions that led to the current state are distributed across system logs that no single interface presents coherently. The BX3 Framework addresses this through the Forensic Ledger (Section **??**), which maintains synchronized, plane-separated records of every decision state, ensuring that a human receiving a Bailout signal has complete diagnostic context available for decision-making.

2.4 Fail-Safe Design in Safety-Critical Systems

The engineering of fail-safe behavior in safety-critical systems provides the most mature body of knowledge for designing systems that behave predictably under unexpected conditions. A fail-safe system is one that transitions to a defined safe state upon failure, as opposed to a fail-dangerous system that continues operating in an unsafe manner. The canonical example is a nuclear reactor scram: upon detecting anomalous conditions, the control system inserts the control rods regardless of any other operational considerations, because the cost of continued operation under uncertainty exceeds the cost of a controlled shutdown.

In autonomous vehicles and cyber-physical systems, fail-safe design has been extensively studied. The Simplex architecture **?** introduced the concept of a *safety controller* that runs in parallel with a high-performance primary controller and can commandeer system execution when the primary controller's proposed action would violate a formally verified safety constraint. This decoupled architecture — performance through the primary controller, safety through an independent, verifiable safety controller — is the direct progenitor of the BX3 three-layer model. In the Simplex model, the safety controller is a software component; in BX3, the analogous function is performed by the Fact Layer, which enforces physical and operational constraints that no Bounds Engine can override, and by the Bailout Protocol, which ensures that unresolved states reach a human safety anchor rather than being resolved by further machine inference.

More recent work **?** extends the fail-safe principle to learning-enabled systems, where the primary controller may be a neural network with unpredictable failure modes. The *Perception Simplex* architecture introduces a fault-tolerant safety layer that can detect when the perception system has entered a state of sensor or model failure and override the mission controller with a safe fallback trajectory. The architectural pattern is consistent: a dedicated, verifiable safety layer that can observe and interrupt the primary operational controller, with provable guarantees about the conditions under which override occurs. BX3's Fact Layer encodes this pattern by enforcing that no physical actuator receives a command that has not been validated against a deterministic model of safe operation. The Sandbox Gate further extends this pattern by requiring digital-twin simulation of any proposed intervention before physical execution is permitted.

A critical distinction separates BX3's fail-safe design from prior work. In the Simplex family of architectures, the safety controller is itself a machine component — albeit a simpler, more formally verifiable one. The override decision is therefore still a machine decision, subject to the same uncertainty and failure modes as the primary controller, though reduced in complexity. In the BX3 Framework, the Bailout Protocol terminates at a human decision-maker, not a machine fallback. The human is not a final-tier machine in the escalation hierarchy; the human is an accountability anchor who receives complete diagnostic context

and makes a genuine, uncomputable judgment call. This reflects the principle articulated in **?**: that machines operating in open environments with genuine uncertainty must ultimately defer to human judgment not because humans are computationally superior, but because accountability for consequential decisions in the real world is a social and legal institution that cannot be instantiated in software.

2.5 The BX3 Framework: A Three-Layer Accountability Architecture

The BX3 Framework synthesizes the preceding threads into a unified architectural model with three immutable functional layers and two enforcement mechanisms. The layers are:

1. **Purpose Layer (Human Accountability Anchor):** The topmost layer consists exclusively of human actors with defined organizational authority. Each node in the system has a Purpose — a specification of the objective it serves — that is authored and owned by a named human. The Purpose Layer never proposes actions autonomously; it authorizes, overrides, or escalates. Every action traceable to the system must have a Purpose Layer owner.

2. **Bounds Engine Layer (Constraint Enforcement):** The middle layer contains autonomous actors that propose actions within the authority granted by the Purpose Layer. The Bounds Engine's role is to evaluate proposed actions against the constraints encoded in its operational mandate and to either approve the action (if all constraints are satisfied), reject the action (if constraints are violated and no alternative satisfies them), or invoke the Bailout Protocol (if the situation exceeds its authority to resolve). The Bounds Engine is explicitly a proposal engine, not an execution engine — it never drives physical actuators directly.

3. **Fact Layer (Deterministic Ground):** The bottom layer encodes the deterministic, verifiable ground truth of the physical or digital system under control. The Fact Layer is the only layer with direct access to physical actuators and data sources. It enforces that all commands reaching actuators conform to the constraints the Fact Layer's deterministic model defines as safe. The Fact Layer is designed to produce identical outputs for identical inputs, enabling formal verification and forensic reconstruction.

The two enforcement mechanisms are the **Sandbox Gate** and the **Bailout Protocol**. The Sandbox Gate requires that any Bounds Engine proposal with physical-world consequences be simulated in a digital twin environment and reviewed by the appropriate Fact Layer gate before physical execution is unlocked. The Bailout Protocol is the subject of this paper: it guarantees that any state the Bounds Engine cannot resolve within its current authority escalates directly to the Purpose Layer human anchor, bypassing all intermediate machine actors, with complete diagnostic context. The system fails upward into human consciousness — it never fails downward into algorithmic chaos.

This architecture satisfies the accountability requirements established by the NIST AI Risk Management Framework and the EU AI Act not through governance documentation, but through architectural enforcement: every autonomous action either originates from a

Purpose Layer authorization or triggers a Bailout that terminates at one. The separation of concerns is not a convention; it is a physical constraint enforced by the layer architecture. The human accountability anchor is not a last resort; it is the required terminal state of every unresolvable exception.

3 The Bailout Protocol

The Bailout Protocol is the runtime mechanism by which the BX3 Framework enforces its upstream accountability guarantee: that every unresolvable exception state propagates to a human accountability anchor, bypassing all intermediate machine actors, with complete diagnostic context, and that the system never fails downward into algorithmic chaos. The protocol is triggered not by machine error but by the Bounds Engine's correct identification of the boundary of its own authority. It is an *authority-exceeded* signal, not a failure signal.

This section specifies the protocol in full formal detail: its state machine, its trigger conditions, its escalation algorithm, its bypass mechanism, and its timeout handling. The specification is complete enough to serve as an authoritative reference for implementation, audit, and formal verification.

3.1 Formal Definitions

We define the following entities and invariants that govern the Bailout Protocol:

Definition 1 (Bounds Engine Node). A Bounds Engine node B is a tuple:

$$B \triangleq \langle C_B, M_B, P_B, \text{BailoutSM}_B \rangle \tag{1}$$

where:

- C_B is the constraint set governing B's operational scope, authored by a Purpose Layer owner.
- M_B is the current mandate of B, a subset of C_B active under the present operational context.
- P_B is the parent node in the system hierarchy; if B is a root node, $P_B = \texttt{null}$.
- BailoutSM_B is the per-instance Bailout Protocol state machine for node B, with state $\sigma_B \in \Sigma$, where $\Sigma = \{\text{IDLE}, \text{BOUNDED}, \text{BAIL_PENDING}, \text{ESCALATING}, \text{HUMAN_ACKNOWLE}$

Definition 2 (Constraint Set). The constraint set C_B is a set of predicate functions over the system state S:

$$C_B = \{c_i : S \to \{\texttt{true}, \texttt{false}\} \mid i = 1, \ldots, n\} \tag{2}$$

A proposed action a is *approved* by B if $\forall c_i \in M_B : c_i(S \mid a) = \texttt{true}$. A proposed action a is *rejected* by B if $\exists c_i \in M_B : c_i(S \mid a) = \texttt{false}$.

Definition 3 (Unresolvable State). A system state $s \in S$ is *unresolvable* by B under mandate M_B if and only if:

$$\text{isUnresolvable}(s, M_B) \triangleq \nexists a : \forall c_i \in M_B : c_i(s \mid a) = \texttt{true} \tag{3}$$

That is, no proposed action exists that satisfies all active constraints. This is the foundational condition for triggering the Bailout Protocol.

Definition 4 (BailoutTrigger). When a trigger condition is satisfied at node B, the Bounds Engine constructs a BailoutTrigger object T:

$$T \triangleq \langle \text{id}, B, s, M_B, \kappa, \psi, \tau, \phi \rangle \tag{4}$$

where:

- id: unique identifier for this bailout event.
- B: the originating Bounds Engine node.
- s: the system state at time of triggering.
- M_B: the active mandate at time of triggering.
- $\kappa \in \mathbb{K}$: the trigger condition class (see Section 3.4).
- ψ: the diagnostic snapshot of B's internal state at time of firing.
- $\tau \in \mathbb{R}^+$: the Unix timestamp at which the trigger was fired.
- $\phi = []$: the propagation chain, initially empty; appended to as T propagates up the hierarchy.

Definition 5 (Trigger Condition Class). The set $\mathbb{K}$ of recognized trigger condition classes is:

$$\mathbb{K} = \{\text{CONSTRAINT_UNCOVERED}, \text{INTERCONSTRAINT_CONFLICT}, \text{FACT_LAYER_VIOLATI} \tag{5}$$

Each class corresponds to a distinct boundary condition of authority (Section 3.4).

Definition 6 (Purpose Layer Owner). The Purpose Layer owner of node B, denoted owner(B), is the named human actor who authored B's mandate M_B and bears accountability for B's outcomes. owner(B) is the sole permissible terminal recipient of a BailoutTrigger.

Definition 7 (Hard Block). When $\sigma_B \in \{\text{BAIL_PENDING}, \text{ESCALATING}, \text{HUMAN_ACKNOWLED}$ the Fact Layer enforces a hard block Γ_B on all physical actuator commands originating from B. Formally:

$$\forall \text{actuator } \alpha : \forall \text{command } a \in A_\alpha \text{ proposed by } B : \Gamma_B(\sigma_B) = \texttt{block}(a) \quad \text{if } \sigma_B \neq \text{IDLE} \wedge \sigma_B \neq \text{RESOLVED} \tag{6}$$

The hard block is monotonic: it is applied at BOUNDED and is not released until RESOLVED. No confidence signal, timer, or business context can override Γ_B.

Definition 8 (Timeout Constants). The Bailout Protocol defines two timeout constants enforced by the Fact Layer:

$$T_{\text{bound}} = 300 \quad \text{(seconds)} \qquad T_{\text{human}} = 7200 \quad \text{(seconds)} \tag{7}$$

T_{bound} is the maximum duration a trigger may remain in the BOUNDED state before automatic transition to BAIL_PENDING. T_{human} is the maximum duration a trigger may remain in HUMAN_ACKNOWLEDGED before the protocol issues a final warning and logs a staleness record. Both constants are set conservatively to allow thorough human review while preventing indefinite escalation.

3.2 The Bailout Protocol State Machine

The Bailout Protocol operates as a deterministic state machine, BailoutSM_B, hard-encoded in the Fact Layer for each Bounds Engine node B. The state machine has six mutually exclusive, collectively exhaustive states:

State σ = IDLE: The default operational state. No trigger is active. The Bounds Engine evaluates and approves or rejects proposed actions within its mandate M_B without involvement of the Bailout Protocol. Transition to BOUNDED occurs when a trigger condition is detected (Section 3.4).

State σ = BOUNDED: A trigger condition has been detected at this node. The Bounds Engine has constructed the BailoutTrigger object T and is in the process of building the full diagnostic context ψ. The Fact Layer hard block Γ_B is applied immediately upon entering this state. The Bounds Engine may not forward any actuator commands from this node. The node remains in BOUNDED for at most T_{bound} seconds. If diagnostic context construction completes within this window, the transition to BAIL_PENDING occurs. If the timeout T_{bound} expires before completion, the Fact Layer forces the transition to BAIL_PENDING with a `CONSTRUCTION_TIMEOUT` flag set in T.

State σ = BAIL_PENDING: Diagnostic context construction is complete. The BailoutTrigger T is sealed and ready for propagation. The Fact Layer maintains the hard block. No machine actor may resolve or dismiss T; the only permissible next state is ESCALATING, in which T is dispatched to the parent node P_B.

State σ = ESCALATING: The BailoutTrigger T has been dispatched to the parent node P_B. The trigger is in transit. The Fact Layer maintains the hard block on the originating node B. The propagation chain ϕ is updated with the identity of B, the timestamp, and the action taken (dispatch). If P_B is a Purpose Layer human anchor (owner(B)), the transition to HUMAN_ACKNOWLEDGED occurs immediately upon receipt. If P_B is another Bounds Engine node, the transition is to the same ESCALATING state (re-dispatch), continuing recursively up the hierarchy until T reaches a human anchor.

State σ = HUMAN_ACKNOWLEDGED: The BailoutTrigger T has been received by owner(B), the Purpose Layer human anchor. The owner has the full diagnostic context and is actively reviewing. The hard block remains in effect. The owner may issue a resolution, request additional information, modify the mandate M_B, or escalate to their own supervisory chain. The system remains in HUMAN_ACKNOWLEDGED until the owner issues a resolution or T_{human} expires. If the owner has not acted within T_{human}, the Fact Layer logs a staleness record and sets a `STALENESS_WARNING` flag, but does not force resolution.

State σ = RESOLVED: The Purpose Layer owner has issued a resolution. The Fact Layer records the resolution in the Forensic Ledger, releases the hard block Γ_B, and the system transitions back to IDLE. This is the terminal state of the Bailout Protocol state machine for this trigger event. For recursive spawning contexts, the resolution may simultaneously serve as the trigger for a new BailoutSM instance at a child node, but for the triggering event itself, RESOLVED is terminal.

3.3 Transition Conditions

Transitions are deterministic events, not timers or confidence thresholds. The complete transition function $\delta : \Sigma \times E \to \Sigma$ is defined as follows, where E is the event set:

Transitions from IDLE:

$$\delta(\text{IDLE}, \texttt{TRIGGER_CONDITION_SATISFIED}) = \text{BOUNDED} \tag{8}$$

Fires when the Bounds Engine detects that a trigger condition $\kappa \in \mathbb{K}$ is satisfied at node B. This is the sole transition out of IDLE.

Transitions from BOUNDED:

$$\delta(\text{BOUNDED}, \texttt{DIAGNOSTIC_COMPLETE}) = \text{BAIL_PENDING} \tag{9}$$

Fires when the Bounds Engine has completed construction of the full BailoutTrigger object T including the diagnostic snapshot ψ.

$$\delta(\text{BOUNDED}, \texttt{TIMER_EXPIRED_BOUND}) = \text{BAIL_PENDING} \tag{10}$$

Fires when elapsed time since entering BOUNDED exceeds T_{bound}. The Fact Layer forces this transition, sets `CONSTRUCTION_TIMEOUT` in T, and logs the timeout event.

Transitions from BAIL_PENDING:

$$\delta(\text{BAIL_PENDING}, \texttt{TRIGGER_DISPATCHED}) = \text{ESCALATING} \tag{11}$$

Fires immediately upon dispatch of T to the parent node P_B. The propagation chain ϕ is updated atomically with this dispatch event.

Transitions from ESCALATING:

$$\delta(\text{ESCALATING}, \texttt{RECEIVED_BY_HUMAN_ANCHOR}) = \text{HUMAN_ACKNOWLEDGED} \quad (12)$$

Fires when the parent node P_B is confirmed to be owner(B) (a Purpose Layer human anchor). This is the terminal propagation transition.

$$\delta(\text{ESCALATING}, \texttt{RECEIVED_BY_MACHINE_NODE}) = \text{ESCALATING} \quad (13)$$

Fires when P_B is another Bounds Engine node. T is re-dispatched to P_{P_B}, the parent of P_B, and the propagation chain ϕ is appended accordingly. This transition may occur repeatedly until a human anchor is reached.

Transitions from HUMAN_ACKNOWLEDGED:

$$\delta(\text{HUMAN_ACKNOWLEDGED}, \texttt{HUMAN_RESOLUTION_ISSUED}) = \text{RESOLVED} \quad (14)$$

Fires when owner(B) issues a resolution. The Fact Layer records the resolution in the Forensic Ledger, releases the hard block, and transitions to IDLE.

$$\delta(\text{HUMAN_ACKNOWLEDGED}, \texttt{TIMER_EXPIRED_HUMAN}) = \text{HUMAN_ACKNOWLEDGED} \quad (15)$$

Fires when elapsed time since entering HUMAN_ACKNOWLEDGED exceeds T_{human}. The Fact Layer logs a `STALENESS_WARNING` in the Forensic Ledger and issues an SMS reminder to owner(B). The state does not change; this transition may re-fire at intervals of T_{human} until a resolution is issued.

Transitions from RESOLVED: There are no outbound transitions from RESOLVED for this trigger instance. The state machine for this instance terminates.

The Fact Layer enforces that no transition not explicitly defined above may occur. This means, for example, that $\delta(\text{BOUNDED}, \texttt{MACHINE_RESOLUTION_ATTEMPTED})$ is undefined and therefore impermissible — the Fact Layer will reject any attempt by a machine actor to resolve a trigger at the BOUNDED state, just as it will reject any attempt to bypass Purpose Layer routing.

3.4 Bail-Out Trigger Conditions

The Bailout Protocol is triggered by the Bounds Engine when it correctly identifies that a condition has arisen that exceeds its resolution authority. The trigger is not an error signal; it is an *authority boundary* signal. The Bounds Engine may be functioning correctly and still fire a bailout, precisely because it has correctly identified that the current situation is outside the scope of its mandate.

A Bounds Engine node B fires a BailoutTrigger when any of the following conditions is satisfied:

T1 (Constraint Uncovered). B receives a request to perform action a, and a is not covered by any constraint in M_B:

$$\nexists c_i \in M_B : c_i \text{ governs } a \tag{16}$$

This condition indicates that the proposed action is outside the scope of the current mandate and requires Purpose Layer authorization.

T2 (Unresolvable State). B detects a system state s for which $\text{isUnresolvable}(s, M_B) = \texttt{true}$ (Definition 3). This condition arises when the environment has moved into a configuration that the current mandate has no approved response to — not because the Bounds Engine failed, but because the mandate has a genuine gap.

T3 (Interconstraint Conflict). B detects a conflict between two or more active constraints $c_i, c_j \in M_B$ such that:

$$\nexists a : c_i(s \mid a) = \texttt{true} \wedge c_j(s \mid a) = \texttt{true} \tag{17}$$

This condition indicates that the mandate contains internally contradictory requirements and requires Purpose Layer disambiguation or mandate revision.

T4 (Fact Layer Violation). B receives a proposed action a that, when validated by the Fact Layer enforcement model, violates a Fact Layer enforcement rule $r \in R_{\text{fact}}$:

$$\text{FactLayer_validate}(a, r) = \texttt{violation} \tag{18}$$

This condition indicates that the Bounds Engine's proposal would breach a deterministic safety constraint that the Fact Layer enforces independently of the Bounds Engine's reasoning.

T5 (Scope Exceeded). B receives a request to perform action a that falls within its technical capability but outside the scope of M_B, as defined by the Purpose Layer owner:

$$a \in \text{capability}(B) \wedge a \notin \text{authorized_scope}(M_B) \tag{19}$$

This condition distinguishes between what B *can* do and what B *is authorized* to do.

T6 (Mandate Modification Request). B receives a request to modify C_B or M_B (including adding, removing, or altering constraints), which it cannot authorize unilaterally:

$$\text{request_type}(r) = \texttt{MANDATE_MODIFICATION} \tag{20}$$

Mandate modifications require Purpose Layer approval and therefore trigger the Bailout Protocol to route the request to $\text{owner}(B)$.

When a trigger condition is satisfied, the Bounds Engine transitions to BOUNDED immediately and begins constructing the BailoutTrigger object T. There is no gating condition, no confidence threshold, and no opportunity for the Bounds Engine to attempt an autonomous resolution before firing. The guarantee of upstream accountability depends on this immediacy: any delay between condition detection and trigger firing creates a window in which an incorrect autonomous action could be taken.

Figure 1: BailoutTrigger Escalation Algorithm.

```
FUNCTION Escalate(T, B):
    phi <- T.phi // Initialize propagation chain
    B_current <- B // Start at originating node

    WHILE B_current != NULL:
        sigma_B_current <- ESCALATING
        phi.append(<B_current, tau_now, DISPATCHED>)
        T.phi <- phi

        IF isHumanAnchor(B_current):
            // B_current is the Purpose Layer owner
            sigma_B_current <- HUMAN_ACKNOWLEDGED
            phi.append(<B_current, tau_now, HUMAN_RECEIVED>)
            T.phi <- phi
            RETURN SUCCESS

        B_parent <- P_B_current

        IF B_parent == NULL:
            // No parent in hierarchy -- log failure and halt
            phi.append(<NULL, tau_now, NO_ANCHOR_FOUND>)
            T.phi <- phi
            RETURN FAILURE // raise HIERARCHY_ERROR: No human anchor in ancestry
                chain

        B_current <- B_parent

    RETURN FAILURE // raise ESCALATION_LOOP_ERROR: Algorithm did not terminate
```

3.5 Escalation Algorithm

The escalation algorithm determines the path a BailoutTrigger takes from the originating Bounds Engine node B to the Purpose Layer human anchor owner(B). The algorithm is defined recursively over the system hierarchy:

The algorithm has the following critical properties:

- **Termination.** The system hierarchy is a rooted tree with a finite depth. Since B_{current} strictly ascends the parent chain on each iteration and the tree has finite depth, the while loop terminates in at most d iterations, where d is the depth of B in the hierarchy. The loop cannot diverge.
- **No intermediate resolution.** The algorithm performs no autonomous reasoning and makes no attempt to resolve the trigger at any machine node. Each machine node in

the chain is a transit point, not a decision point. This is the architectural commitment that the bypass mechanism (Section 3.6) enforces.

- **Single destination.** The algorithm routes to exactly one human anchor: owner(B), the Purpose Layer owner of the originating node. It does not clone the trigger to multiple recipients, does not broadcast to a team, and does not select a recipient based on availability or load. This ensures a single accountable decision-maker.
- **Complete traceability.** The propagation chain ϕ records every node in the hierarchy through which T passed, with a timestamp and action at each hop. This record is included in the Forensic Ledger entry for this bailout event and is the solution to the abstraction leakage problem described in Section 2.

The Fact Layer enforces the escalation algorithm's invariants at the architectural level. A machine node cannot refuse to forward a BailoutTrigger, cannot select a different parent node, and cannot substitute a machine fallback for the human anchor. The algorithm is not a policy that machine actors may choose to follow or deviate from; it is a physical constraint of the layer architecture.

3.6 Bypass Mechanism: Why the Bailout Protocol Bypasses All Machine Actors

The most critical property of the Bailout Protocol is that it bypasses all machine actors in its escalation path. When a BailoutTrigger reaches a machine node in the hierarchy during escalation, that node does not attempt to resolve the trigger. It forwards the trigger to its own parent and stops. The trigger does not ask the machine node's opinion, does not request a confidence-weighted assessment, and does not allow the machine node to substitute its own resolution.

This bypass is not a convention or a software convention. It is a structural property enforced by the layer architecture. To understand why the bypass is architecturally necessary, consider the failure modes that arise if machine actors are permitted to in the escalation path:

Failure Mode 1: Autonomous Override. If a machine node in the escalation path could resolve a BailoutTrigger, it would become a de facto autonomous override of human accountability. The node could apply its own reasoning, propose a resolution, and either dismiss the trigger or substitute its own action — precisely the pattern that the Bailout Protocol is designed to prevent. The upstream accountability guarantee would be voided by a silent machine override that the human never sees.

Failure Mode 2: Confidence-Based Routing. If machine nodes could assess a BailoutTrigger and decide whether to escalate based on their own confidence, the system would introduce a probabilistic component into an architectural safety mechanism. A machine node that is highly confident in its own resolution would suppress the trigger and proceed autonomously — the same failure mode that the Simplex architecture's formal verification requirements are designed to prevent. The confidence signal is internal to the machine layers and is not a permitted input to the escalation routing decision.

Failure Mode 3: Abstraction Leakage Amplification. If machine nodes could respond to a BailoutTrigger, the human accountability anchor receiving the escalation would lose additional context with each hop, as each intervening machine node transforms, filters, or summarizes the diagnostic information before passing it up. This amplification of abstraction leakage would undermine the forensic standard that the protocol is designed to maintain. The bypass mechanism ensures that the diagnostic context ψ that reaches the human anchor is the original context constructed at the originating node, not a transformed version.

The bypass mechanism is enforced by the Fact Layer at the state machine level. When σ_B = ESCALATING, the Fact Layer state machine for node B rejects any inbound event from a machine actor that attempts to resolve the trigger, sets the node's σ_B back to ESCALATING, and logs the override attempt as a `BYPASS_VIOLATION` event in the Forensic Ledger. This is the same deterministic enforcement mechanism that enforces all Fact Layer constraints, and it carries the same weight.

The architectural implication is that the Bailout Protocol does not merely *permit* escalation to a human anchor — it *prevents* all alternative termination points. The system cannot accidentally terminate a bailout at a machine actor, because the Fact Layer will not accept a machine resolution at any state other than RESOLVED, and RESOLVED can only be entered via the HUMAN_ACKNOWLEDGED → RESOLVED transition initiated by the human anchor. The state machine is a physical enforcement mechanism, not a workflow suggestion.

This design reflects the principle articulated in Section 2: that accountability for consequential decisions in the real world is a social and legal institution that cannot be instantiated in software. The bypass mechanism ensures that the Bailout Protocol maintains this distinction at runtime, under all conditions, including system stress, high workload, and component degradation.

3.7 Timeout Handling

The Bailout Protocol defines two timeout constants, T_{bound} and T_{human}, that govern the maximum duration a trigger may spend in the BOUNDED and HUMAN_ACKNOWLEDGED states, respectively. Timeout handling is designed to prevent indefinite stalling at either the diagnostic construction phase or the human review phase, while preserving the human's full authority to review at their own pace.

BOUNDED State Timeout (T_{bound}). The BOUNDED state is the phase during which the Bounds Engine constructs the complete diagnostic context ψ for the BailoutTrigger. In most cases, diagnostic context construction completes quickly — typically within seconds. However, in complex scenarios involving nested system state, multiple sensor inputs, and cross-plane diagnostic data, construction may take longer.

If T_{bound} (300 seconds, 5 minutes) expires before construction is complete, the Fact Layer forces the transition to BAIL_PENDING with the `CONSTRUCTION_TIMEOUT` flag set in T. The trigger proceeds to escalation with whatever diagnostic context was available at the time of timeout. The Purpose Layer owner receiving the bailout is informed that the diagnostic context is incomplete and may request additional information before issuing a resolution.

The purpose of T_{bound} is not to penalize the Bounds Engine for slow construction, but to prevent a pathological scenario in which a system under stress (where bailout triggering is most likely) could stall indefinitely in the diagnostic phase due to degraded performance. The timeout ensures that the escalation phase begins within a bounded time regardless of system conditions.

HUMAN_ACKNOWLEDGED State Timeout (T_{human}). The HUMAN_ACKNOWLEDGED state is the phase during which the Purpose Layer owner reviews the BailoutTrigger and its diagnostic context and issues a resolution. Human review is intentionally unbounded in the base protocol — there is no scenario in which a machine actor should force a resolution over the objection or non-action of the human accountability anchor. The T_{human} timeout (7200 seconds, 2 hours) exists to address a different failure mode: the scenario in which the human anchor has not engaged with the bailout due to notification failure, absence, or oversight.

When T_{human} expires, the Fact Layer:

1. Logs a `STALENESS_WARNING` in the Forensic Ledger for this bailout event.
2. Issues an SMS reminder to owner(B) via the INBOX routing system, with a direct link to the bailout context and a summary of the triggering condition.
3. Transitions the state machine back to HUMAN_ACKNOWLEDGED (the state does not change; only a warning is issued).

The T_{human} timeout may re-fire at intervals of T_{human} seconds, with each firing logged as a separate `STALENESS_WARNING` entry. The Fact Layer continues to log warnings and issue reminders until the human anchor issues a resolution or the system is formally halted by an authorized administrator.

There is no automatic resolution. Even under repeated staleness warnings, the Fact Layer will not accept a resolution from any actor other than the confirmed human anchor. If the human anchor is permanently unreachable (e.g., due to incapacitation), the escalation algorithm's hierarchy traversal provides for a `HIERARCHY_ERROR` condition (Algorithm 1, line 20), which surfaces the failure to the next available human anchor in the chain of organizational authority. This fallback is the only permitted deviation from the single-anchor rule, and it is triggered only by the absence of a human anchor in the ancestry chain, not by any machine actor's preference.

The timeout handling design reflects the principle that human review is the mandatory terminal state of the Bailout Protocol and that timeouts exist to ensure the system does not stall — not to substitute machine judgment for human judgment. The guarantees of the Bailout Protocol hold regardless of timeout behavior, because timeouts never permit autonomous resolution.

4 Implementation in the BX3 Framework

The Bailout Protocol is not a safety module added to an existing autonomous system. It is the runtime expression of the BX3 Framework's upstream accountability guarantee: that every exception state at any Bounds Engine node propagates to a human accountability

anchor, bypassing all machine intermediaries. Making this guarantee architecturally enforceable rather than merely policy-level requires the Bailout Protocol to be integrated into all three functional layers at their boundaries. This section specifies that integration: how the *Purpose Layer* anchors human accountability at the top, how the *Bounds Engine* detects and fires bailout signals, how the *Fact Layer* enforces the hard block and maintains the forensic record, and how the state machine operates as a deterministic extension of the *Fact Layer*.

4.1 Purpose Layer: Human Accountability Anchor

The *Purpose Layer* is the only layer that may legitimately terminate a Bailout Protocol signal. This is a definitional commitment, not a design choice among alternatives. The *Purpose Layer* is defined by its capacity for genuine accountability — the ability to bear responsibility for a decision in a way that is legally, socially, and organizationally enforceable. A machine actor, regardless of its sophistication, cannot bear responsibility in this sense. It can optimize, recommend, and defer — but it cannot be held accountable the way a named human can. Therefore, only a *Purpose Layer* actor may receive and resolve a Bailout signal.

In the BX3 three-layer model, every node in a recursive system carries a *Purpose* authored and owned by a named human principal $h(n)$. This reference is stored in the node's Purpose layer worksheet and is immutable for the node's operational lifetime. The immutability of $h(n)$ is the structural property that terminates the escalation chain and makes the upstream accountability guarantee unconditional. No machine actor — including the node's own Bounds Engine, any parent node, or any recursively spawned subordinate — may modify $h(n)$ without explicit human authorization recorded as a Forensic Ledger entry.

The *Purpose Layer* serves a dual function in the Bailout Protocol:

1. **Authorization provenance.** The *Purpose Layer* establishes the authoritative chain from which all legitimate system action derives. Every *Bounds Engine* action traces back to a *Purpose Layer* authorization. If a bailout fires, the *Purpose Layer* owner receives the complete chain of prior authorizations and proposed actions, enabling informed judgment rather than requiring them to reconstruct context from fragments.

2. **Resolution authority.** The *Purpose Layer* owner has the authority to override the bailout, modify the node's constraints, reassign the node to a different *Bounds Engine*, halt the system, or escalate to their own supervisory chain. No machine actor has this range of resolution options. This is what distinguishes a genuine accountability anchor from a supervisory review layer.

The key architectural property is that the *Purpose Layer* is not merely *available* as an escalation target. It is structurally the *only* permissible terminal state for any unresolved exception. The BX3 upstream accountability guarantee is enforced by the layer architecture itself: the *Bounds Engine* may not terminate a bailout signal at another machine actor, and the *Fact Layer* hard-blocks any command that would bypass *Purpose Layer* routing for an active exception condition. This is not a privilege level that can be reconfigured — it is a structural property of the layer model.

4.2 Bounds Engine: Triggering Bail-Out

The *Bounds Engine* is the layer that detects and fires bailout signals. This is a direct consequence of its defining role: the *Bounds Engine* evaluates proposed actions against encoded constraints and either approves, rejects, or escalates. When the situation exceeds its resolution authority, the *Bounds Engine* does not guess, does not default to a safest-known action, and does not recursively attempt autonomous resolution. It fires the Bailout Protocol.

The trigger conditions that activate the Bailout Protocol are formally defined as follows. A *Bounds Engine* node B fires a bailout signal when any of the following conditions hold:

1. **Uncovered state:** B detects a system state s that is not covered by any constraint in C_B, where C_B is the constraint set defined for B's current operational scope, and $isUnresolvable(s, C_B)$ evaluates to true — meaning no rule in C_B can produce an approved action for state s.

2. **Inter-constraint conflict:** B encounters a conflict $conflict(c_1, c_2)$ between two or more constraints $c_1, c_2 \in C_B$ such that no action a satisfies both constraints simultaneously.

3. **Fact Layer violation:** B receives a proposed action a that, when evaluated against the *Fact Layer* validation model, would violate a deterministic enforcement rule $r \in R_{fact}$, where R_{fact} is the set of enforcement rules maintained by the *Fact Layer*.

4. **Scope exceeded:** B receives a request to perform an action a that is not within the scope of C_B.

5. **Mandate modification request:** B receives a request to modify its own constraint set C_B, which it cannot authorize and which therefore requires *Purpose Layer* approval.

These are not error codes in the conventional software engineering sense. They are *boundary conditions of authority*: states in which the *Bounds Engine* has correctly identified that it lacks the information or authorization to proceed, and that further progress requires a different kind of judgment — one that only a human accountability anchor can provide.

When a trigger condition fires, the *Bounds Engine* constructs a `BailoutTrigger` object capturing the complete diagnostic context: the triggering event, the node state at time of firing, the constraint conflict or boundary condition encountered, the *Bounds Engine* confidence level, and any proposed resolution that was attempted and failed. This object is the unit of propagation in the bailout signal chain.

Critically, firing the Bailout Protocol does not indicate that the *Bounds Engine* has made an error. The *Bounds Engine* may be functioning correctly, following its constraints faithfully, and still encounter a condition its constraints cannot resolve. The Bailout Protocol is not a failure signal; it is an *authority-exceeded* signal. It fires precisely because the *Bounds Engine* has correctly identified the boundary of its mandate — and correctly declined to cross that boundary autonomously.

The *Bounds Engine* also enforces a suspension property: upon firing, the originating agent is atomically suspended from further action on the affected task. This prevents a second

agent or process from racing past the bailout to execute the same action on a different thread while the escalation is in progress. The suspension is enforced by the *Fact Layer* upon receipt of the `BailoutTrigger`, not by the *Bounds Engine* itself — ensuring that the suspension cannot be suppressed by a compromised *Bounds Engine*.

4.3 Fact Layer: Hard Block and Forensic Ledger

The *Fact Layer* serves two distinct functions in the Bailout Protocol: it provides the deterministic enforcement mechanism that prevents premature or unauthorized continuation of machine action during a bailout, and it maintains the Forensic Ledger that records the complete bailout event chain for post-hoc accountability review.

The *Fact Layer* enforces a *hard block* on any physical actuator command associated with a live bailout signal. When a `BailoutTrigger` is in the `fired` or `escalating` state, the *Fact Layer* refuses to forward any *Bounds Engine* command to physical actuators. This is not a software conditional:

```
if (bailoutActive) { block() }  // SOFT BLOCK -- bypassable
```

It is a deterministic enforcement rule in the *Fact Layer*:

```
on receiving cmd to actuator:
    if bailout_state != RESOLVED:
        HARD_BLOCK  // structurally enforced, not conditionally checked
        return
    else:
        forward cmd
```

The block cannot be overridden by a higher-confidence *Bounds Engine* proposal, a changed business context, a timeout, or any machine actor. The block holds until the `BailoutTrigger` reaches `human_review` and the *Purpose Layer* owner issues an explicit resolution. The block is a property of the *Fact Layer*'s state machine, not a conditional check that can be bypassed.

The Forensic Ledger records every bailout event in real time across three planes of the nine-plane DAP structure (Plane 4: Accountability; Plane 6: Temporal Sequencing; Plane 8: Compliance Posture). A dedicated `BailoutEventRecord` schema extends the standard event record:

- **Trigger condition type:** A machine-readable code identifying the category of trigger condition (e.g., `UNCOVERED_STATE`, `INTERCONSTRAINT_CONFLICT`, `FACT_LAYER_VIOLATION`, `SCOPE_EXCEEDED`).
- **Originating node:** The identifier of the *Bounds Engine* node at which the bailout was fired, enabling reconstruction of the full propagation path.
- **Diagnostic snapshot:** A complete capture of the *Bounds Engine* state at time of firing, including constraint state, sensor readings, prior actions, and the specific conflict or boundary condition encountered.

- **Propagation chain:** An ordered, timestamped list of every node through which the `BailoutTrigger` propagated, with the action taken at each hop.
- **Resolution record:** The resolution issued by the *Purpose Layer* owner, including the action taken, any mandate modifications issued, and the explicit reasoning provided.
- **Chain integrity hash:** The SHA-256 hash of this ledger entry, chained to the hash of the preceding entry, ensuring tamper evidence across the full bailout record.

The append-only, hash-chained structure ensures that any modification to a prior entry — whether accidental or deliberate — breaks the hash chain at the point of modification and is detectable in $O(1)$ by any verifier performing a chain integrity check. The *Fact Layer* performs this check automatically after every event write and triggers a bailout if a break is detected, ensuring that ledger integrity is itself subject to the upstream accountability guarantee.

4.4 Bailout Protocol as Runtime Expression of the Upstream Accountability Guarantee

The upstream accountability guarantee — that systems built on the BX3 Framework *fail upward into human consciousness, never downward into algorithmic chaos* — is a structural property of the framework, guaranteed by the layer architecture. The Bailout Protocol is the runtime mechanism by which this guarantee is effectuated in real-time execution.

Consider the contrast with conventional autonomous systems. In a conventional system, human oversight is typically a supervisory layer that monitors agent outputs and can intervene upon detecting a problem. The pipeline continues until a threshold is crossed. This creates two failure modes that BX3 eliminates.

First, the system can fail dangerously while awaiting threshold crossing if the threshold is miscalibrated or the failure mode was unanticipated. Second, human oversight in the conventional model is reactive rather than mandatory: it depends on a human noticing a problem in time to intervene, with all the attentional and cognitive limitations that entails.

The BX3 upstream accountability guarantee eliminates both failure modes by making human oversight the *mandatory terminal state of every unresolvable condition*, not an optional intervention layer. The Bailout Protocol fires when the *Bounds Engine* identifies a condition it cannot resolve — not when a human notices a problem. The human is inserted into the loop by the architecture, not by their own vigilance. This transforms the human's role from reactive monitor to active, authoritative decision-maker engaged precisely at the moments when the system's autonomous capability has reached its boundary.

The Bailout Protocol therefore does not merely *enable* upstream accountability. It *enforces* it architecturally. The distinction is between a system designed to route exceptions to a human *if possible*, and a system architecturally incapable of terminating an unresolvable exception at any machine actor. The BX3 Framework is the latter. The Bailout Protocol is the mechanism that makes this architectural commitment operative at runtime.

The five pillars of BX3 collectively establish the conditions under which the upstream accountability guarantee applies. Loop Isolation, Recursive Spawning, Spatial Firewall, and

Sandbox Gate address exception conditions anticipated at design time and resolvable within the machine-only execution graph. The Bailout Protocol addresses the complementary remainder — conditions not anticipated, that exceed the operating range of any machine actor, and that cannot be resolved within the machine-only execution graph. The completeness of this coverage is what makes the upstream accountability guarantee unconditional: it is not that *most* exceptions reach $h(n)$, but that *every* exception reaches $h(n)$, because the trigger condition fires on any condition outside $R(n)$ and the escalation path has no machine-actor terminus by architectural constraint.

4.5 State Machine as Fact Layer Extension

The Bailout Protocol state machine is hard-encoded in the *Fact Layer*. The *Fact Layer* does not merely enforce the output of the Bailout Protocol; it defines the protocol's state machine itself. This is a critical architectural commitment: the state transitions of the Bailout Protocol are deterministic, not probabilistic, and are enforced by the same mechanism that enforces all *Fact Layer* constraints. There is no separate Bailout Protocol module that can be modified independently of the *Fact Layer*.

The state machine defines the following states:

1. `idle`: Default state. No trigger is active. The *Bounds Engine* operates normally within its constraint set.

2. `fired`: A trigger condition has been detected. The *Bounds Engine* has constructed the `BailoutTrigger` and the *Fact Layer* has received it. All physical actuator commands from the affected node are hard-blocked.

3. `escalating`: The `BailoutTrigger` has been dispatched to the parent node in the propagation chain. The trigger is in transit. The *Fact Layer* maintains the hard block on the originating node.

4. `human_review`: The `BailoutTrigger` has arrived at the *Purpose Layer* human anchor. The *Purpose Layer* owner has the diagnostic context and is actively reviewing the bailout. The hard block remains in effect.

5. `resolved`: The *Purpose Layer* owner has issued a resolution. The *Fact Layer* records the resolution in the Forensic Ledger, releases the hard block, and the system transitions back to `idle`.

The state transition function is:

$$\delta : \mathcal{S} \times \mathcal{E} \to \mathcal{S} \tag{21}$$

where $\mathcal{S} = \{idle, fired, escalating, human_review, resolved\}$ and $\mathcal{E}$ is the set of events defined by the transition predicates in Table 1. Each transition is atomic and deterministic: given the same state and event, the same next state is produced.

Table 1: Bailout Protocol State Transitions as Deterministic *Fact Layer* Rules

From	**Event (Transition Predicate)**	**To**
idle	$trigger_condition() = \top$	*fired*
fired	Diagnostic context construction complete	*escalating*
fired	$T_{escalate}$ timeout expires	*escalating*
escalating	Parent node is *Purpose Layer* anchor, trigger received	*human_review*
escalating	Parent node is not *Purpose Layer* anchor	*escalating* (re-dispatch)
human_review	*Purpose Layer* owner issues resolution	*resolved*
human_review	T_{human} timeout expires	*escalating* (re-notify)
resolved	Post-processing complete	*idle*

The transition *human_review* → *resolved* requires a *Purpose Layer* authorization payload: a resolution authored by a named human and digitally signed with their *Purpose Layer* credentials. The *Fact Layer* verifies this payload before accepting the transition. No machine actor can produce a *Purpose Layer* authorization payload, so no machine actor can resolve a bailout unilaterally.

The three state invariants maintained by the *Fact Layer*'s enforcement architecture are:

$$\text{INV}_1 : \forall n \in \text{nodes},\ \text{state}(n) \in \mathcal{S} \setminus \{\text{escalating}\} \iff \neg\text{active_bailout}(n). \tag{INV-1}$$

INV-1 is enforced by the *Fact Layer*'s pre-commit hook: a node may enter a non-escalating state only when the Bailout Monitor confirms that no active propagation is in flight. The state and the active-bailout flag are updated atomically, preserving the equivalence.

$$\text{INV}_2 : \text{state}(n) = \text{fired} \implies \text{active_bailout}(n) \wedge \delta(\text{fired}, e_{\text{timeout}}) = \text{escalating}. \tag{INV-2}$$

INV-2 follows from the *Fact Layer*'s enforcement of the transition relation. Any node in `fired` has an active bailout, and the only terminal transition defined for this state with an enabled event is to `escalating`. The *Fact Layer* enforces this transition compulsorily upon receipt of e_{timeout}; no machine actor may interpose.

$$\text{INV}_3 : \text{state}(n) = \text{human_review} \implies \exists\ \text{Purpose_layer_auth}(n) \wedge \text{directive}(n) \in \{\texttt{ACK}, \texttt{RESOLVE}, \texttt{REJECT} \tag{INV-3}$$

INV-3 establishes that a node enters `human_review` only when a Purpose-layer-authorized directive is present and recorded. This prevents the state machine from entering the human-review state based on machine-generated signals alone; the state is only reached upon receipt of an authenticated *Purpose Layer* communication.

The hard block is monotonic with respect to the state machine: the block is applied at *fired* and is not released until *resolved*. There is no state in which the hard block is conditionally applied based on confidence level, elapsed time, or business context. This monotonicity property is what makes the Bailout Protocol a structurally reliable safety mechanism rather than a configurable policy that can be inadvertently weakened.

The *Fact Layer* maintains the state machine for every active `BailoutTrigger` across the entire system topology, enabling the framework to support recursive spawning patterns in which child nodes inherit the Bailout Protocol state machine from their parent, with the same states, transitions, and enforcement rules. The protocol is not a single-instance mechanism — it is a replicated, per-node state machine that propagates upward through the hierarchy when triggered.

The combination of the state machine specification in the *Fact Layer*, the monotonic hard block on actuator commands, the three-plane Forensic Ledger recording, the *Bounds Engine* trigger conditions, and the three state invariants constitutes a complete, architecturally enforced implementation of the Bailout Protocol. The upstream accountability guarantee is not maintained by convention, by operational policy, or by the reliability of any individual component. It is maintained by the layer architecture itself, enforced at the *Fact Layer*, and triggered by the *Bounds Engine*. The system fails upward, without exception, and with a complete forensic record of every step in the chain.

5 Guarantees and Correctness Properties

The Bailout Protocol, as specified in Sections 3 and 4, makes three non-negotiable guarantees about the behavior of any BX3 node that encounters an unresolvable condition: **SAFETY** — the node cannot collapse autonomously into uncontrolled action; **LIVENESS** — the exception condition eventually reaches a human accountability anchor; and **ACCOUNTABILITY** — the human who receives the exception is always identifiable. These guarantees are not operational policies. They are architectural properties enforced by the layer structure, the deterministic state machine, and the tamper-evident Forensic Ledger. This section establishes each guarantee formally, defines the conditions under which it holds, and provides a proof sketch demonstrating why the design ensures the guarantee is satisfied in all permitted execution paths.

5.1 Formal Framework for Guarantees

We work within the formal model established in Sections 3 and 4. Let $\mathcal{B}$ denote the Bailout Protocol state machine for a given Bounds Engine node B, with state space $\Sigma = \{\text{IDLE}, \text{BOUNDED}, \text{BAIL_PENDING}, \text{ESCALATING}, \text{HUMAN_ACKNOWLEDGED}, \text{RESOLVED}\}$. Let $\text{owner}(B)$ denote the designated Purpose Layer human anchor for B, and let $h(n)$ denote the immutable human anchor reference stored at node creation. The hard block Γ_B is enforced by the Fact Layer as a monotonic constraint: $\Gamma_B(\sigma) = \texttt{block}$ for all $\sigma \in \{\text{BOUNDED}, \text{BAIL_PENDING}, \text{ESCALATING}, \text{HUMAN_ACKNOWLEDGED}\}$ and $\Gamma_B(\text{RESOLVED}) = \texttt{release}$.

We prove each guarantee by demonstrating that the Bailout Protocol's state machine and layer architecture together eliminate all execution paths that would violate the guarantee.

5.2 GUARANTEE 1: Safety — No Autonomous Collapse

Guarantee (Safety). *When a BailoutTrigger is active at node B, no physical actuator command originating from B is forwarded to any actuator until the trigger reaches the RESOLVED state. No machine actor can transition the trigger to RESOLVED.*

The Safety guarantee addresses the core failure mode identified in Section 3: autonomous drift, in which a system continues to act without human review because no threshold was crossed and no escalation was triggered. The Bailout Protocol eliminates this failure mode by converting the absence of a machine-resolvable path into a mandatory hard block on all actuator commands, preventing any machine actor from advancing the system state in the affected domain.

Theorem 1 (Safety). *For any node B and any active BailoutTrigger T at B, no command issued by B reaches any physical actuator while T is in any state $\sigma \in \{BOUNDED, BAIL_PENDING, ESCA$ Furthermore, no machine actor can produce a state transition from any of these states to RESOLVED.*

Proof Sketch. The proof has two parts.

Part (a) — No actuator command propagates during an active trigger. By Definition 7 (Hard Block), the Fact Layer enforces $\Gamma_B(\sigma) = \texttt{block}$ for all $\sigma \in \{\text{BOUNDED}, \text{BAIL_PENDING}, \text{ESCALATING}$ The Fact Layer is the sole authority over physical actuator interfaces: no actuator receives any command that has not passed through the Fact Layer's pre-commit hook. When Γ_B evaluates to `block`, the hook refuses to forward the command and logs a `BLOCKED_ACTUATOR_ATTEMPT` event. This enforcement is monotonic: there is no transition in the state machine (Table 1) that releases Γ_B while σ is in the active trigger states. The only release transition is $\delta(\text{HUMAN_ACKNOWLEDGED}, \texttt{HUMAN_RESOLUTION_ISSUED}) = \text{RESOLVED}$, which requires a Purpose Layer authorization payload authored by a human. No machine actor can produce this payload. Therefore, no actuator command from B reaches any actuator while T is active. □

Part (b) — No machine actor can resolve the trigger. The state transition HUMAN_ACKNOWLEDGED – RESOLVED is defined exclusively by the event `HUMAN_RESOLUTION_ISSUED`, which carries a Purpose Layer authorization payload digitally signed by the human anchor. The Fact Layer's authorization ledger gate rejects any transition attempt that does not carry a verified Purpose Layer credential. No Bounds Engine, no parent node, no system administrator, and no external machine actor possesses a valid Purpose Layer credential for owner(B). The set of actors capable of producing `HUMAN_RESOLUTION_ISSUED` is exactly $\{\text{owner}(B)\}$, a singleton containing one human. Therefore, no machine actor can unilaterally resolve a BailoutTrigger. □

The Safety guarantee holds regardless of the trigger condition class $\kappa \in \mathbb{K}$, the depth of the originating node in the spawning hierarchy, the number of intermediate machine nodes in

the propagation chain, or the load or health state of the system. The hard block is enforced by the Fact Layer independently of any reasoning or confidence assessment by any machine actor. The only path out of a safe (blocked) state is through a human-issued resolution, which the Forensic Ledger records as a Purpose Layer authorized entry with the principal identifier of the accountable human.

Invariant 1 *Safety Invariant For all B, for all t, if a BailoutTrigger T is active at B at time t, then $\Gamma_B(\sigma_B(t)) = \texttt{block}$.*

Invariant 1 is enforced by the Fact Layer pre-commit hook as a first-class constraint. Any attempt to advance the state machine without satisfying this invariant is rejected, and the attempted violation is logged as a `SAFETY_CONSTRAINT_VIOLATION` event in the Forensic Ledger.

5.3 GUARANTEE 2: Liveness — Eventual Human Contact

Guarantee (Liveness). *For every BailoutTrigger T fired at any node B in the spawning hierarchy, if the system hierarchy is structurally sound (i.e., every node has a defined parent or is a Purpose Layer anchor), then T eventually reaches the HUMAN_ACKNOWLEDGED state, or the system halts with a logged HIERARCHY_ERROR.*

The Liveness guarantee addresses the concern that an escalation could stall indefinitely — caught in an infinite loop of machine-to-machine propagation, never reaching a human. The Bailout Protocol's liveness property follows from the structure of the spawning hierarchy and the algorithmic constraints on the escalation path.

Theorem 2 (Liveness). *Let $\mathcal{H}$ be the spawning hierarchy rooted at h, the top-level Purpose Layer anchor. If every node in $\mathcal{H}$ has a defined parent pointer P_n (or is a Purpose Layer anchor with $P_n = \texttt{null}$), then for any BailoutTrigger T fired at any descendant node $B \in \mathcal{H}$, the escalation algorithm (Algorithm 1) terminates in at most $d(B)$ iterations, where $d(B)$ is the depth of B in $\mathcal{H}$, and the terminal state is either HUMAN_ACKNOWLEDGED (if the anchor is reachable) or the algorithm raises HIERARCHY_ERROR (if the ancestry chain is broken).*

Proof Sketch. The proof relies on two structural properties of the BX3 spawning hierarchy: finite depth, and the non-circularity of parent pointers.

Property 1 — Finite depth. The BX3 spawning hierarchy is a rooted tree: each node n has exactly one parent P_n, except the root Purpose Layer anchor h which has $P_h = \texttt{null}$. The depth function $d(n)$ is the number of edges from h to n, which is well-defined and finite for all nodes in the tree. The BX3 Framework enforces a maximum spawn depth $D_{\max}$ at provisioning time, which provides an additional practical bound on the number of escalation hops.

Property 2 — Strict ascent. Algorithm 1 (line 15: $B_{\text{current}} \leftarrow B_{\text{parent}}$) strictly ascends the parent chain on each iteration. A node's parent is always at strictly lower depth: $d(P_n) <$

$d(n)$. Therefore, the parent relation defines a strict well-ordering over the nodes of depth greater than zero.

Termination argument. Consider the sequence of nodes visited by Algorithm 1:

$$B = n_0, \quad n_1 = P_{n_0}, \quad n_2 = P_{n_1}, \quad \ldots$$

By Property 2, this sequence is strictly ascending in depth: $d(n_0) > d(n_1) > d(n_2) > \cdots$. By Property 1, depth is bounded below by $d(h) = 0$. Therefore, the sequence terminates in at most $d(B)$ steps at some node n_k with $d(n_k) = 0$, which by definition is the root Purpose Layer anchor h. The loop cannot diverge, cannot cycle, and cannot stall because each iteration advances to a strictly shallower node and the depth space is finite. □

The liveness guarantee depends on one structural condition: that the spawning hierarchy is sound. A broken parent pointer — which could arise from a bug in the spawning implementation or deliberate corruption of the hierarchy — would trigger a HIERARCHY_ERROR condition, which Algorithm 1 detects, logs explicitly, and surfaces to the next available human in the organizational chain. The error is not silently absorbed. This fallback is the only permitted deviation from the single-anchor rule, and it is triggered only by the absence of a human anchor in the ancestry chain, never by any machine actor's preference.

Invariant 2 *Escalation Path Existence For every node B in the spawning hierarchy, there exists a finite sequence of parent pointers $P_{n_0} = B, P_{n_1}, P_{n_2}, \ldots, P_{n_k} = h$ such that each P_{n_i} is either a Bounds Engine node or a Purpose Layer anchor, and h is a verified human principal.*

Invariant 2 is established at node provisioning time. When a new node is spawned, the Fact Layer verifies that the parent pointer P_B is defined and points to a valid node in the hierarchy before the spawn is committed. A node with an undefined or invalid parent pointer is not activated. This prevents the introduction of orphaned nodes — nodes with no escalation path — into the operational hierarchy.

5.4 GUARANTEE 3: Accountability — Human Always Identifiable

Guarantee (Accountability). *For every BailoutTrigger T that reaches the RESOLVED state, the Forensic Ledger contains an immutable record identifying the principal of the human anchor who issued the resolving directive. No ledger entry recording a human resolution can be created without the explicit authorization of that human.*

The Accountability guarantee ensures that the Safety and Liveness guarantees have teeth: escalation to a human is not merely achieved but is verifiably attributable to a specific, named human principal. This is what converts the Bailout Protocol from a safety mechanism into an accountability infrastructure — one that can satisfy regulatory requirements for decision attribution.

Theorem 3 (Accountability). *For any BailoutTrigger T that reaches RESOLVED, the Forensic Ledger entry $\mathcal{L}_T$ contains a non-null field* `principal` *identifying exactly one human*

principal $p \in \mathcal{P}$, where $\mathcal{P}$ is the set of authorized Purpose Layer principals in the system. Furthermore, $\mathcal{L}_T$ is timestamped, SHA-256 chain-sealed, and cannot be modified after commitment without breaking the chain.

Proof Sketch. The proof traces the lifecycle of a BailoutTrigger through the Forensic Ledger and shows that the principal field is both mandatory and authenticated at every step.

Step 1 — Trigger creation. When a trigger condition fires at node B, the Fact Layer creates a Forensic Ledger entry recording the triggering event. This entry is written by the Fact Layer — the sole write authority for the Forensic Ledger — and includes the originating node B, the trigger condition class κ, the timestamp τ, and the diagnostic context. At this stage, the `principal` field is `null` because no human has yet acted.

Step 2 — Escalation propagation. As T propagates up the hierarchy (Algorithm 1), each hop generates a Forensic Ledger entry recording the transit event: the node that forwarded T, the timestamp, and the action taken (`DISPATCHED`). The propagation chain ϕ is updated atomically with each hop. No entry in the propagation chain contains a non-null `principal` field, because all transit nodes are machine actors.

Step 3 — Human receipt. When T arrives at the Purpose Layer human anchor owner(B), the Fact Layer creates a `HUMAN_RECEIVED` entry, recording the verified identity of owner(B). The identity is established by the Fact Layer's verification of the Purpose Layer authentication credential accompanying the receipt acknowledgment. This is the first point at which a human identity is recorded for this trigger.

Step 4 — Resolution and closure. When owner(B) issues a resolution directive, the Fact Layer verifies the Purpose Layer credential, records the directive in the Forensic Ledger, and closes the trigger by creating the RESOLVED entry. The `principal` field of the RESOLVED entry is set to the verified identity of owner(B). This field is mandatory: the Fact Layer's authorization ledger gate rejects any RESOLVED transition that does not carry a verified Purpose Layer credential with a non-null principal identifier.

Step 5 — Chain integrity. Every entry in the above sequence is SHA-256 chain-sealed at the time of commitment. The `principal` field is part of the sealed content of the RESOLVED entry. Modifying this field after commitment would change the entry's content hash, breaking the chain at that entry and all subsequent entries. The tamper-evidence property of the Forensic Ledger ensures that any such modification is detectable in $O(n)$ verification time, requiring no access to internal system state. □

The Accountability guarantee is structurally dependent on two immutable constraints: the immutability of the $h(n)$ anchor reference (which prevents substitution of a different human identity at resolution time), and the Fact Layer's exclusive write authority over the Forensic Ledger (which prevents a compromised Bounds Engine from fabricating a RESOLVED entry with a fabricated principal). Both of these protections are enforced by the BX3 layer separation and are not reliant on any individual component's reliability.

Invariant 3 *Principal Attribution For every BailoutTrigger T that reaches RESOLVED, the Forensic Ledger entry for T satisfies* `principal` $\neq$ `null` *and* `principal` $\in \mathcal{P}$*, where $\mathcal{P}$ is the set of authorized Purpose Layer principals.*

5.5 Corollaries and Interaction Effects

The three guarantees are not independent — they interact in ways that strengthen the overall accountability posture.

Corollary 1 (No Silent Override). Combining Safety and Accountability: because no machine actor can resolve a trigger (Safety, Part b), and every resolution is recorded with a human principal (Accountability), there is no execution path by which a bailout event is resolved without an identifiable human's explicit authorization. The system cannot silently override or absorb a bailout event.

Corollary 2 (Complete Escalation Path). Combining Liveness and Accountability: because every trigger reaches a human (Liveness), and every resolution identifies that human (Accountability), the full escalation path of any bailout event is attributable end-to-end. The system provides a complete, auditable chain from the triggering condition to the human who addressed it.

Corollary 3 (No Privilege Elevation). Combining Safety with the bypass mechanism: because the bypass mechanism prevents machine actors from attempting resolution at any intermediate node (Section 3.6), and Safety prevents any actuator command from propagating during an active trigger, a compromised or malfunctioning intermediate machine node cannot exploit a bailout event to elevate its own privileges or execute unauthorized actions. The trigger bypasses the node entirely, and the node's actuators remain blocked.

5.6 Verification and Audit

The three guarantees are independently verifiable by an external auditor without access to internal system state or cryptographic keys. Verification proceeds entirely from the Forensic Ledger.

Safety verification. Retrieve the Forensic Ledger entries for all BailoutTrigger events in a given operational window. For each trigger, confirm that: (a) no `ActuatorCommand` entry appears in the ledger between the BOUNDED entry and the RESOLVED entry for the same trigger ID; and (b) the RESOLVED entry carries a Purpose Layer credential with a non-null `principal` field. If condition (a) holds for all triggers, the Safety invariant (Invariant 1) has been upheld for the window. If condition (b) holds for all triggers, the Accountability invariant (Invariant 3) has been upheld.

Liveness verification. For each trigger in the window, retrieve the propagation chain ϕ from the RESOLVED entry. Verify that the chain terminates at a Purpose Layer anchor (i.e., the final entry in ϕ has `HUMAN_RECEIVED`). Verify that the chain depth is finite and consistent with the known hierarchy depth of the system. Identify any trigger that terminates with HIERARCHY_ERROR and confirm that the error was surfaced to an operational administrator.

Accountability verification. For each RESOLVED entry, verify: (a) the `principal` field is non-null and the principal is a member of the registered Purpose Layer principal set $\mathcal{P}$; and (b) the SHA-256 chain integrity holds from the preceding entry through this entry. Perform a full chain verification on the ledger as a whole using the $O(n)$ forward-pass algorithm.

These verification procedures are entirely retrospective: they require only a copy of the Forensic Ledger and access to the principal registry. No live system access, no debugging interface, and no cooperation from the running system are required. An auditor with appropriate access can determine, in a single deterministic pass, whether the three guarantees held for any operational window.

The combination of architectural enforcement (the state machine, the hard block, the bypass mechanism), tamper-evident logging (the Forensic Ledger), and independent verifiability (the audit procedures above) constitutes the complete accountability guarantee of the Bailout Protocol. The system does not merely claim to maintain human accountability — it produces evidence of that accountability that any authorized party can verify without relying on the system's own self-reporting.

6 Limitations and Future Work

This section acknowledges the open limitations of the Bailout Protocol as currently specified, describes the conditions under which each limitation is most acute, and identifies three productive directions for future research and engineering work that would strengthen the protocol's applicability in larger-scale and higher-stakes deployments.

6.1 Limitations

The following limitations are not implementation defects correctable by refinement within the current specification. They are structural properties of the protocol's design that impose real constraints on its applicability.

6.1.1 Human Response Latency

The Bailout Protocol's liveness guarantee (Theorem 2) depends on the assumption that the Purpose Layer human anchor will engage with the bailout signal and issue a resolution within a time interval compatible with the operational requirements of the system. In slow-cycle domains—regulatory compliance review, strategic planning, multi-party contract negotiation—this assumption is well-grounded. In real-time cyber-physical domains, it is not.

Consider an autonomous agricultural system performing sub-second irrigation adjustments in response to microclimate sensor data. If the Bounds Engine fires a bailout because it encounters an unresolvable constraint state (for example, a water rights conflict between co-located parcels under different ownership), the protocol requires the human anchor to review the complete diagnostic context and issue a resolution before the Fact Layer will release the hard block on actuators. The human anchor may be unavailable, traveling, or cognitively engaged in an unrelated task. The T_{human} timeout (2 hours, as specified in Section 3.7) is architecturally conservative precisely because it does not impose a deadline on human judgment; but that conservativeness means the system may be stalled for a duration that is unacceptable in time-sensitive operational contexts.

The limitation is fundamental: the protocol trades real-time performance for accountability guarantee. For any deployment in which human review latency exceeds the operational

time budget, the protocol either must be supplemented with a safe fallback state that preserves the hard block but maintains minimum viable operation, or the protocol's applicability must be scoped away from that operational tier. Neither workaround is built into the current specification.

6.1.2 Adversarial Conditions

The protocol is designed to operate correctly under conditions of component degradation, environmental novelty, and operator error. It is not designed to operate correctly under conditions of active adversarial interference with the escalation path itself.

An adversarial agent or external actor with access to the system hierarchy could, in principle, interfere with bailout signal delivery by severing or corrupting the parent pointer chain, by denying network connectivity to the human anchor's notification channel, or by impersonating the human anchor to receive and resolve a bailout with fraudulent authority. The protocol's forensic ledger provides retrospective detection of such interference (the `HIERARCHY_ERROR` and `BYPASS_VIOLATION` markers are logged when propagation fails or override attempts are detected), but it does not prevent interference in real time.

The practical implication is that the protocol's security guarantees hold under the assumption that the system hierarchy and communication channels are trusted infrastructure. For deployment in adversarial network environments—for example, multi-agent systems operating across organizational trust boundaries without a shared identity and access management infrastructure—additional integrity mechanisms (authenticated parent chains, signed escalation messages, multi-factor human anchor verification) would be required before the protocol's guarantees hold. These mechanisms are not specified in the current design.

6.1.3 Scalability

The Bailout Protocol's correctness guarantees are proved for a system topology consisting of a single Bounds Engine node firing a single BailoutTrigger upward along a linear parent chain to a single human anchor. This model is adequate for small-scale deployments and for formal analysis. It is not adequate for multi-agent systems with dozens or hundreds of concurrent nodes, each capable of firing bailouts independently.

Under high node density, the following failure modes emerge:

- **Anchor saturation.** A single human anchor assigned as `owner()` to n concurrent nodes may receive n simultaneous BailoutTriggers, each requiring genuine review. Human cognitive bandwidth is not a scalable resource. The protocol currently provides no mechanism for distributing anchor assignments across a pool of qualified human reviewers, nor any mechanism for prioritizing signals by operational criticality.

- **Hierarchy depth.** In wide organizational hierarchies (common in enterprise multi-agent deployments), the distance from a leaf node to its designated Purpose Layer anchor may be large. A bailout at a leaf in a 10-level hierarchy must transit all 10 intermediate machine nodes, each of which may introduce propagation latency or failure points. The protocol's correctness proofs do not cover this case.

- **Cascading bailout storms.** Under system-wide stress conditions (network partition, sensor degradation, or coordinated environmental shock), multiple nodes may fire bailouts simultaneously. The protocol does not specify any behavior for coordinating concurrent bailout signals at the same or different anchor nodes, nor any graceful degradation policy when bailout volume exceeds human review capacity.

These scalability failure modes are known and acknowledged. They do not invalidate the protocol's correctness guarantees under its stated assumptions; they define the boundaries of those assumptions for deployment planning.

6.1.4 Exploitation Risk

The Bailout Protocol creates a single, well-defined choke point through which all unresolvable exception states must pass: the Purpose Layer human anchor. A malicious actor who can trigger the protocol at will—by crafting inputs that satisfy trigger condition T1 (constraint uncovered) or T5 (scope exceeded) repeatedly—can subject the human anchor to a denial-of-service pattern in which their attention is consumed by a high volume of low-stakes bailout signals, rendering them unavailable for genuine high-stakes exceptions.

This exploitation risk is distinct from the adversarial conditions described above. In the adversarial case, the attacker targets the escalation infrastructure itself. In the exploitation case, the attacker targets the human anchor's attention, using the protocol's mandatory escalation requirement as the attack vector. The protocol's `SCOPE_EXCEEDED` trigger condition (T5) is particularly exposed: an attacker with the ability to submit inputs outside the authorized scope of a node can fire a bailout at will for any node whose bounds are insufficiently narrow.

The current specification does not include any mechanism for rate-limiting bailout signals, for distinguishing genuine scope-exceeding events from adversarial input manipulation, or for automated pre-screening of bailout signals before they reach the human anchor. These are significant operational security gaps that must be addressed before deployment in adversarial environments.

6.2 Future Work

Three directions for future research and engineering work are identified as the most productive paths from the current specification to a deployment-ready protocol.

6.2.1 Formal Verification

The correctness properties stated in Section **??** are presented as proof sketches with explicit assumptions. A full formal verification of the Bailout Protocol state machine using a mechanized proof assistant (Coq, Isabelle, or TLA+) would transform these sketches into machine-checked theorems.

The verification effort would target three specific claims: (1) that the state machine transition function δ has no undefined transitions for any valid state-event pair in $\Sigma \times E$; (2) that the escalation algorithm terminates in at most d steps for any node at depth d in

a finite rooted hierarchy; and (3) that the hard block Γ_B is monotonic with respect to the state ordering, meaning the block is never released before the `RESOLVED` state is reached.

Mechanized verification is particularly important for the bypass mechanism (Section 3.6), where the claim that no machine actor may resolve a trigger at any state other than `RESOLVED` is load-bearing for the entire accountability architecture. A mechanized proof that this property holds for all valid state transitions would close the principal remaining gap between the informal correctness argument and a formal guarantee.

Additionally, formal verification of the protocol's composition with the Recursive Spawning Protocol (RSP) would establish that the upstream accountability guarantee holds across nested spawn chains, not only for single-level nodes. The RSP introduces parent chain depth as a variable that affects the distance a BailoutTrigger must travel to reach a human anchor, and the interaction of this depth variable with the timeout constants T_{bound} and T_{human} requires careful analysis.

6.2.2 Adaptive Timeout

The current specification uses fixed timeout constants $T_{\text{bound}} = 300\,\text{s}$ and $T_{\text{human}} = 7200\,\text{s}$. These constants were chosen conservatively to accommodate thorough human review under normal operating conditions. They are not adapted to the operational context of the deployment.

An adaptive timeout extension would adjust timeout values based on three signals: (1) the operational criticality of the affected node (as defined by the Purpose Layer owner at provisioning); (2) the current backlog of pending bailout signals at the designated human anchor; and (3) historical data on human response times for comparable trigger condition classes. For a node operating in a non-critical decorative context, longer timeouts with lighter staleness warnings may be appropriate. For a node operating in a safety-critical context, shorter timeouts with escalation to a secondary anchor may be required.

The adaptive timeout mechanism must preserve the protocol's core invariant: no autonomous resolution is ever permitted regardless of timeout behavior. Adaptation should affect only the warning cadence and secondary anchor escalation timing, not the conditions under which the `RESOLVED` state may be entered. A productive formal framework for this extension is a timeout policy function $\theta : \mathbb{K} \times \mathbb{C} \times \mathbb{N} \to \mathbb{R}^+$ where $\mathbb{K}$ is the trigger condition class, $\mathbb{C}$ is the node criticality tier, and $\mathbb{N}$ is the current anchor backlog, producing a context-appropriate timeout value.

6.2.3 Distributed Human Anchor Pools

The scalability limitations described in Section 6.1.3 could be substantially mitigated by replacing the single-anchor model with a distributed human anchor pool: a defined set of qualified human reviewers, any one of whom may receive and resolve a bailout signal, with an allocation policy that distributes signals across the pool based on workload, expertise, and availability.

The distributed anchor pool extension requires three additional components. First, an **anchor discovery protocol** that resolves the question of which human anchors are available and qualified at the time a bailout fires, given that the Purpose Layer owner designated at

provisioning may be unavailable. Second, an **allocation policy** that selects among available anchors using criteria that preserve the single-accountable-decision-maker property: exactly one anchor must receive each trigger, and that anchor must have genuine authority over the affected node's mandate. Third, a **distribution audit mechanism** that records in the Forensic Ledger which anchor received each bailout, enabling retrospective accountability reconstruction in cases where the allocation decision is questioned.

The extension is architecturally compatible with the current protocol: the escalation algorithm (Algorithm 1) resolves a single parent node per iteration, and the distributed pool extension replaces the static owner(B) designation with a dynamic lookup against the pool registry. The state machine transitions and the hard block mechanism are unchanged. This makes the distributed anchor pool a natural layer-3 extension rather than a structural modification of the protocol's core guarantees.

7 Conclusion

The Bailout Protocol specifies the runtime mechanism by which the BX3 Framework's upstream accountability guarantee is enforced in real-time multi-agent execution. The contribution is architectural and formal: the protocol defines a deterministic state machine, triggered by the Bounds Engine's correct identification of the boundary of its own authority, that propagates every unresolvable exception state directly to a named human accountability anchor, bypassing all intermediate machine actors, with complete diagnostic context, and with a tamper-evident forensic record of the entire propagation chain.

The protocol's correctness properties were established formally. The Safety property (Theorem 1) guarantees that no machine actor may issue a terminal resolution to any Bailout-Trigger: the state machine has no transition from any state other than `HUMAN_ACKNOWLEDGED` to `RESOLVED`, and the Fact Layer hard block Γ_B is monotonic, applied at `BOUNDED` and not released until `RESOLVED`. The Liveness property (Theorem 2) guarantees that every valid trigger event reaches a human anchor within a bounded number of escalation steps, because the system hierarchy is a finite rooted tree and the escalation algorithm ascends the parent chain strictly. The Accountability property (Theorem 3) guarantees that the human anchor identified at resolution time is the same human anchor designated at the node's provisioning time, established by the immutable parent chain and enforced by the Fact Layer's deterministic state machine.

These three properties jointly constitute the upstream accountability guarantee as a runtime invariant, not merely a design aspiration. The guarantee is not maintained by the reliability of any individual component, by operational policy, or by the vigilance of human monitors. It is maintained by the layer architecture of the BX3 Framework, enforced at the Fact Layer, and triggered by the Bounds Engine's boundary conditions of authority. The system fails upward into human consciousness—it never fails downward into algorithmic chaos.

The Bailout Protocol is the runtime expression of this guarantee. The BX3 Framework's three-layer model—Purpose Layer for human accountability, Bounds Engine for constrained execution, Fact Layer for deterministic enforcement—is the architectural substrate that makes the protocol structurally necessary, not merely recommended. Every mechanism

in the protocol maps onto exactly one layer: the trigger condition detection maps onto the Bounds Engine's boundary evaluation function; the hard block maps onto the Fact Layer's deterministic enforcement; the escalation terminus maps onto the Purpose Layer's exclusive resolution authority. This mapping is not a design pattern. It is a structural necessity. Removing any layer causes the corresponding guarantee to collapse.

The Bailout Protocol was specified in sufficient formal detail to serve as an authoritative reference for implementation, audit, and formal verification. The state machine, trigger conditions, escalation algorithm, bypass mechanism, and timeout handling were each defined precisely. Limitations were acknowledged transparently: human response latency in real-time contexts, adversarial vulnerability of the escalation path, scalability under high node density, and exploitation risk from deliberate trigger manipulation. Three productive directions for future work were identified: mechanized formal verification, adaptive timeout policy, and distributed human anchor pools.

The broader contribution is to the problem of accountability in multi-agent AI systems. As autonomous agents are deployed in higher-stakes environments—agricultural operations, infrastructure management, healthcare, financial systems—the question of who is responsible when something goes wrong is no longer a philosophical or regulatory abstraction. It is an engineering requirement. The Bailout Protocol demonstrates that mandatory human accountability as an architectural fallback is not merely desirable but formally achievable: that a deterministic, auditable, structurally enforced protocol can guarantee that every unresolvable exception state in a multi-agent system terminates in human judgment, with complete forensic context, without probabilistic components, and without relying on human vigilance as the primary safety mechanism. This is the contribution that the BX3 Framework was designed to deliver.

8 Related Work

The Bailout Protocol synthesizes and extends ideas from four intersecting research traditions.

8.1 Accountability in Multi-Agent AI Systems

Multi-agent AI systems introduce accountability challenges not present in single-agent deployments. When multiple autonomous agents coordinate to decompose and execute complex directives, responsibility for outcomes becomes distributed across the agent network in ways that are opaque, non-linear, and legally ambiguous [?]. Traditional accountability frameworks presuppose a fixed hierarchy of human agents with delineated authority; they were designed for deterministic, fully-specified systems. Modern AI agents introduce states that are not exhaustively enumerable, whose failure modes are neither fully predictable nor fully auditable post hoc.

The BX3 Framework addresses this through a structured accountability architecture in which every node carries an immutable parent pointer and a designated human anchor. The parent pointer defines the chain of delegation; the human anchor closes the loop by ensuring that some named human principal is architecturally reachable from every node. This is an

architectural compulsion: the Bailout Protocol does not permit any machine actor to serve as a terminal resolution point for an unresolved exception.

8.2 Human-in-the-Loop Design

Human-in-the-loop (HITL) design positions human agents as integral components of AI-driven workflows. The canonical motivation is epistemic: AI systems cannot reliably reason about novel situations or value trade-offs that require human judgment [?]. A secondary motivation is institutional: many decisions have legal, financial, or social consequences that cannot be delegated to a machine regardless of empirical performance [?]. Conventional HITL architectures treat human oversight as a high-tier option; the Bailout Protocol's bypass property ensures human oversight is the *only* permissible terminus for every unresolved exception.

8.3 Fail-Safe Design in Safety-Critical Systems

Classic fail-safe designs in aerospace, nuclear, and medical systems rely on defined safe states that the system enters upon anomaly detection, awaiting human review before resuming operation. These designs are structurally similar to the Bailout Protocol's ESCALATING state. The protocol extends the classical fail-safe concept by addressing the case where the anomaly detection mechanism may be operating under computational constraints that prevent correct classification. The trigger condition fires on observational boundary violation rather than on error classification.

8.4 Formal Verification of Distributed Protocols

Recent work on formal verification of agentic systems [?] demonstrates that formal methods can provide push-button correctness guarantees. The Bailout Protocol's state machine $\mathcal{B} = (Q, q_0, \Sigma, \rightarrow, Q_F)$ is designed to be amenable to similar verification: the state space is finite and explicitly enumerated, the transition relation is deterministic, and the invariant properties (SAFETY, LIVENESS, ACCOUNTABILITY) are formally stated and provable.

8.5 Relationship to Tiered Agentic Oversight (TAO)

Tiered Agentic Oversight (TAO) [?] demonstrates that hierarchical supervision among specialized agents can reduce error propagation. TAO routes to human oversight as a high-risk escalation pathway; the Bailout Protocol makes this routing unconditional and architectural. The key distinction is the bypass property: in TAO, a parent agent may absorb an exception; in the Bailout Protocol, absorption at any intermediate machine tier is a protocol violation.

8.6 Relationship to the BX3 Framework

The Bailout Protocol is the runtime expression of BX3's upstream accountability guarantee. Within the BX3 three-layer model, the *Purpose Layer* layer provides the human anchor $h(n)$, the *Bounds Engine* layer enforces the trigger condition and manages the transition to

ESCALATING, and the *Fact Layer* layer enforces the transition relation and maintains the Forensic Ledger. Together, the three layers operationalize BX3's design principle: systems fail upward into human consciousness, never downward into algorithmic chaos.

9 Conclusion

The Bailout Protocol addresses a fundamental structural gap in the accountability architecture of multi-agent AI systems: the absence of a mandatory, architecturally enforced mechanism that guarantees every unresolved exception reaches a designated human principal who can issue a binding resolution directive.

The core contribution is the deterministic commitment-before-handoff property: an agent that triggers bailout must commit to its current state before any human-facing signal is transmitted, and must hold that state until a unanimous acknowledgment is received. This property ensures human oversight is never bypassed, even under concurrent execution, network partitions, or component failure. It is not a convention — it is an architectural constraint enforced by the Fact Layer pre-commit hook, the immutable parent-pointer chain, and the append-only Forensic Ledger.

The protocol's operation within the BX3 Framework demonstrates that the upstream accountability guarantee is not merely a design principle but a mechanically enforceable correctness invariant. The *Purpose Layer* layer's human anchor is the only permissible terminus for every unresolved exception; the *Bounds Engine* layer's trigger condition fires on observational boundary violation; the *Fact Layer* layer's pre-commit hook prevents any state transition that violates the transition relation; and the Forensic Ledger provides the immutable evidentiary record that makes every protocol run attributable and auditable.

The protocol's limitations — human response latency in real-time domains, adversarial integrity under active network interference, and scalability under concurrent high-density node operation — are documented and characterized. Future work includes formal verification in a temporal logic or proof assistant, adaptive timeout estimation using historical human-response data, and distributed human-anchor pools with load balancing.

The Bailout Protocol is released as part of the BX3 Framework. Its formal specification and reference implementation constitute a foundation for verifiable, auditable, human-centered AI infrastructure in any domain where the consequences of autonomous action justify mandatory human review.